



Lab 12: Panoptic segmentation

MCHA4400

Semester 2 2024

Introduction

In this lab, you will prepare training data and use it to fine tune a panoptic segmentation model to detect instances of ducks (🦆). This can be used as a feature detector for Scenario 6 in Assignment 2.



GenAI Tip

You are strongly encouraged to explore the use of Large Language Models (LLMs), such as GPT, Gemini or Claude, to assist in completing this activity. If you do not already have access to an LLM, bots are available to use via the UoN Mechatronics Slack team, which you can find a link to from Canvas. If you are unsure how to make the best use of these tools, or are not getting good results, please ask your lab demonstrator for advice.

1 Preparation

Basic proficiency with Python and PyTorch is assumed (from ENGG3300 or otherwise). It is recommended that you use a Python [virtual environment](#)¹. If you are not familiar with virtual environments or have only used older versions of Python, refer to [this guide](#) for more information.

Tasks

- Create and activate a Python virtual environment within the `lab12/python` directory. This can be done from an IDE such as VS Code or PyCharm, or via the terminal as shown below.

Terminal

```
nerd@basement:~/MCHA4400/lab12$ cd python
nerd@basement:~/MCHA4400/lab12/python$ python3 -m venv .venv
nerd@basement:~/MCHA4400/lab12/python$ source .venv/bin/activate
(.venv) nerd@basement:~/MCHA4400/lab12/python$
```



Info

The terminal indicates that the virtual environment is active by displaying the name of the environment in parentheses at the start of the prompt. The virtual environment can be exited by entering `deactivate` at the terminal prompt.

- Install the required Python packages within the virtual environment. These are included within the provided `requirements.txt` for your convenience.

Terminal

```
(.venv) nerd@basement:~/MCHA4400/lab12/python$ python3 -m pip install --upgrade pip
(.venv) nerd@basement:~/MCHA4400/lab12/python$ python3 -m pip install -r requirements.txt
[Package installing intensifies]
(.venv) nerd@basement:~/MCHA4400/lab12/python$
```

¹This is essentially required from Python version 3.11 and above, which implements [PEP 0668](#).

2 Data

A set of annotated images are included in the `data/images` directory that may be used for training the duck detector. These annotated images were made using a simple colour/shape-based detector in OpenCV. The process involves converting the image to HSV color space, applying color-based thresholding², performing morphological operations to clean up the mask³, detecting contours⁴, and then analysing each contour for size, shape, and position to determine if it represents a roughly circular object with a yellow hue and high colour saturation.

Duck instance masks are drawn with a unique hue with maximum saturation and brightness in the annotated image, over a background that is a darkened version of the original image for context. This format allows for easy visual identification of individual duck instances while preserving some context from the original scene.

These annotated images are not perfect, in particular, note that

- Non-yellow parts of the duck (e.g., beak/eyes) may be excluded from the instance masks.
- Ducks near the edge of the image are not correctly detected.
- Ducks with small apparent size in the image are not correctly detected.
- Overlapping ducks are erroneously assigned the same instance masks.

Tasks

- a) Select a suitable subset of annotated images (and their corresponding original images) from `data/images` and copy them into `data/train` to form the training set.



Tip

In order of descending priority, where possible, avoid selecting annotated frames with

1. Multiple ducks within the same instance mask.
2. False positives (e.g., knots in the floorboards).
3. False negatives (e.g., undetected ducks in the distance).
4. Ducks with missing non-yellow parts.

- b) (optional) If more high-quality training data is needed, consider using image editing software to manually improve the instance mask quality in the provided annotated images. In particular,
- a) Split overlapping instances into separate masks with unique hue values per instance mask.
 - b) Extending existing instance masks to cover non-yellow regions (e.g., beak/eyes).
 - c) Painting in undetected ducks using unique hue values.
 - d) Painting out false positives in black.

²via `cv::cvtColor()` and `cv::inRange()`

³via `cv::erode()` and `cv::dilate()`

⁴via `cv::findContours()`

3 Fine-tuning and model export

Given an input image, the fine-tuned Mask2Former model produces two main outputs:

- **Class queries logits:** These are raw, unnormalized scores for each class (in our case, background and duck) for each query. The model predicts these scores for a fixed number of queries, which can be thought of as potential object detections.
- **Mask queries logits:** These are raw, unnormalized scores for each pixel, indicating whether it belongs to the object represented by each query. The dimensions of these logits are typically smaller than the input image size due to the model's internal downsampling.

After fine-tuning the model, we also include the following post-processing steps into the model before exporting it for deployment:

- **Class query logit normalisation:** We apply a softmax function to the class queries logits. This converts the raw scores into probability-like values, ensuring they sum to 1 across classes for each query. The resulting class scores represent the confidence of each query belonging to each class.
- **Mask interpolation:** The mask queries logits are upsampled from 128×128 to 384×384 using bilinear interpolation.
- **Mask query logit normalisation:** We apply a sigmoid function to the interpolated mask queries logits. This converts the raw scores into probability-like values between 0 and 1 for each pixel, indicating the confidence of that pixel belonging to the object represented by each query.

Including these post-processing steps within the exported model reduces the computational overhead during inference, as these operations are optimised and executed within the model's computational graph. Additionally, it simplifies the deployment process by providing a single, self-contained model that produces ready-to-use outputs, reducing the potential for errors in implementing separate post-processing steps.

Tasks

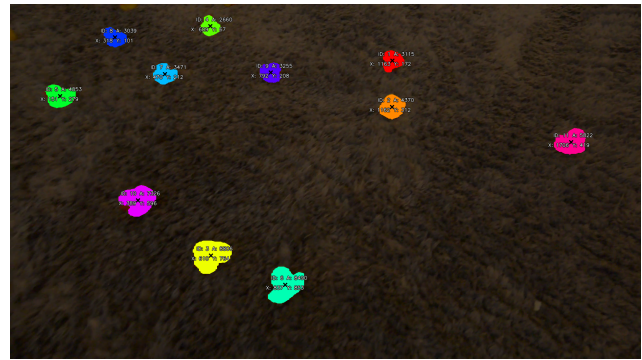
- Run `python/duck_finetune.py` to fine tune a pretrained Mask2Former model on the training data set prepared in `data/train`.
- (optional) Consider using data augmentation (e.g., colour jitter, scale/shift/rotation) to synthetically increase the size of the training set (input images and their instance masks) to make the duck detector more robust to variations in lighting, scale, and orientation.
- Run `python/duck_export_with_postprocessing.py` to convert the fine-tuned Mask2Former model into TorchScript (`duck_with_postprocessing.pt`) and Open Neural Network eXchange (ONNX) format (`duck_with_postprocessing.onnx`) for later deployment.

4 Deployment and evaluation

We will deploy and evaluate the trained duck detector on either `libtorch`, which is the C++ runtime library for PyTorch, or `onnxruntime`, which is a cross-platform runtime for ONNX models. An example of the output of the duck detector for a given input image is shown in Figure 1.



(a) Input image.



(b) Output image with mask, centroid and area information for detected duck instance.

Figure 1: Example of input and output images for duck detector.

Tasks

- a) If you have not already done so from Lab 1, install `libtorch` (via the `pytorch` package) and `onnxruntime` via homebrew.

Terminal

```
nerd@basement:~/MCHA4400/lab12$ brew install pytorch onnxruntime
```



MSYS2 Warning

libtorch and onnxruntime are not currently supported under MSYS2.



Note

The homebrew packages for `libtorch` and `onnxruntime` are precompiled for CPU deployment. If you wish to deploy on GPU, you will need to compile/install an appropriate version of `libtorch` or `onnxruntime` with CUDA/TensorRT support. Consult the official documentation for your chosen runtime for GPU deployment instructions.

- b) Select either `libtorch` or `onnxruntime` as the runtime by uncommenting the relevant lines in `src/main.cpp` to create an instance of `DuckDetectorTorch` or `DuckDetectorONNX`, respectively.
- c) Configure and build the `lab12` application within VS Code or via the terminal.

Terminal

```
nerd@basement:~/MCHA4400/lab12$ cmake -G Ninja -B build && cd build
nerd@basement:~/MCHA4400/lab12/build$ ninja
```

- d) Complete the implementation of `DuckDetectorBase::postprocess` in `src/DuckDetectorBase.cpp` to compute the centroid and area of each detected duck instance and overlay these on the output image similar to Figure 1b.

- e) Evaluate the performance of the duck detector on the provided evaluation video and export the output video.

Terminal

```
nerd@basement:~/MCHA4400/lab12/build$ ninja && ./lab12 ../data/duck.MOV --export
```